



中国科学技术大学
University of Science and Technology of China



USTC, CHINA
ADSLAB

Typst template for School of Computer Science and Technology, USTC

Continuously Improving...

Quaternijkon

ADSLAB, USTC

2024-10-05

NNS

ANSS

NNS

ANSS

- Brute Force Approach
- Tree-Based Approach
 - KD-tree
 - Ball-tree
 - R-tree
 - M-tree

KD 树是每个叶子节点都为 k 维点的二叉树。所有非叶子节点可以视作用一个超平面把空间分割成两个半空间。节点左边的子树代表在超平面左边的点，节点右边的子树代表在超平面右边的点。

KD-tree 最邻近搜索

- 1 **start** 从根节点开始向下递归:
- 2 **if** 点在超平面的左边:移动到 **左子节点**
- 3 **else**: 移动到 **右子节点**
- 4 **once** 到达 **叶节点**, 将当前叶节点设置为 "**当前最好节点**"
- 5 **for all**:
- 6 **if** 目前所在点比目前最佳点更靠近输入点
- 7 | | 将其变为目前最佳点。

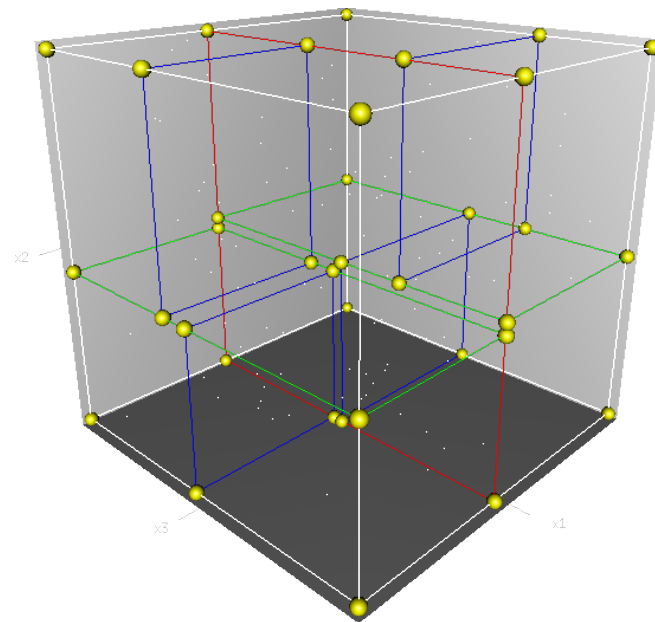


图 1 一个三维 k-d 树。第一次划分（红色）把根节点（黄色）划分成两个节点，然后它们分别再次被划分（绿色）为两个子节点。最后这四个子节点的每一个都被划分（蓝色）为两个子节点。因为没有更进一步的划分，最后得到的八个节点称为叶子节点。

Ball-tree 每个节点代表一个超球 (ball), 由一个中心点和一个半径定义, 包含空间中一部分数据点。非叶节点包含其子节点的超球范围信息, 用于剪枝和加速查询。叶节点包含少量的实际数据点, 由于直接计算。

Ball-tree 最邻近搜索

测试点为 T , 从 root 开始采用 DFS 检查节点。维护一个最大优先优先级队列, 用 Q 表示目前遇到的 k 个最近点。在每个节点 B 上:

- 1 | if $\text{distance}(T, B) > \max \text{ distance in } Q$, ignore B .
- 2 | if B is leaf node, for all node in B : update queue.
- 3 | B is internal node, searching the child whose center is closer to T .

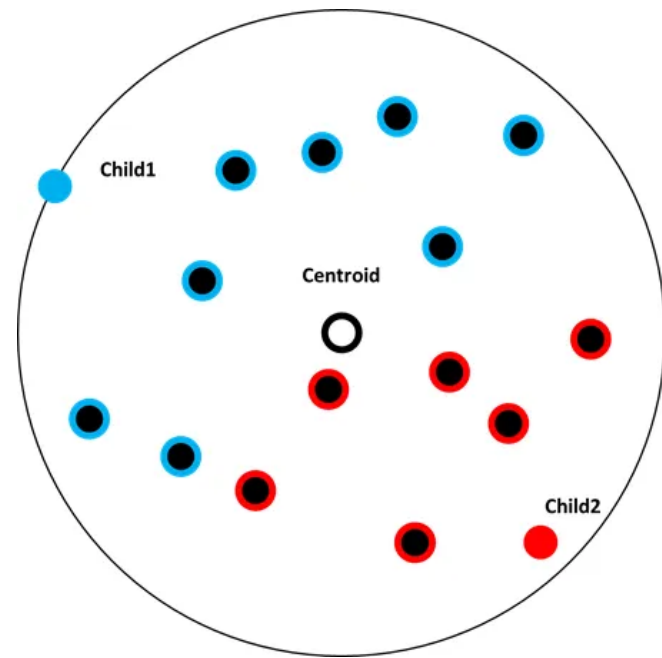


图 2 计算当前节点所有数据的质心 (centroid), 找到离质心最远的两个数据点, 标记为 child1 和 child2。确定 child1 和 child2 后, 将当前节点的所有数据根据距离划分: 距离 child1 较近的数据归入 child1 对应的子节点, 距离 child2 较近的数据归入 child2 对应的子节点。

R-tree 的每个节点代表一个最小边界矩形 (Minimum Bounding Rectangle, MBR), 包含若干个子节点或数据对象。非叶节点包含子节点的 MBR, 用于构建树的层次结构。叶节点包含实际的空间数据对象。R-tree 是一棵平衡树, 所有的叶节点在同一层。

R-tree 最邻近搜索

- 1 初始化: 设置最小优先队列, 将根节点加入优先队列。
- 2 DFS:
- 3 从队列中取出最小距离的节点。
- 4 if 叶节点, 计算其中对象与查询点的实际距离, 更新最近邻信息。
- 5 if 内部节点, 将其子节点根据最小距离加入优先队列。

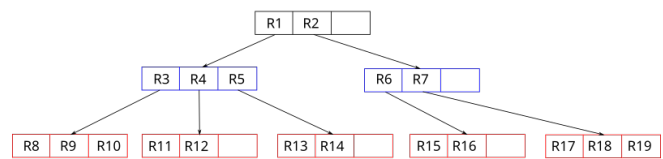
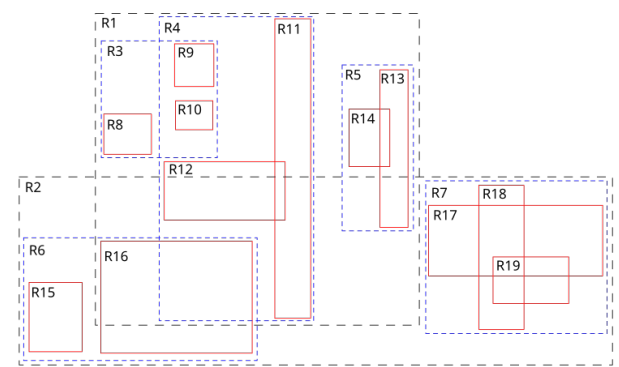


图 3 根据待插入对象的 MBR, 与树中节点的 MBR 计算重叠和扩展量, 选择需要最小化 MBR 扩展的路径。将对象的 MBR 插入选定的叶节点。如果叶节点超过了最大容量, 则需要分裂成两个节点, 并调整父节点的 MBR。

NNS

ANSS

